

SOAP-to-REST Gateway — Multi-Module Proxy Integration Service

Code-generated REST proxy adapters bridging a legacy SOAP backend to modern REST consumers — Java 17 · Spring Boot 3.3 · Apache CXF 3.5 · JAXB/JAXWS · XSLT code generation · EhCache 3

Java 17

Spring Boot 3.3

Apache CXF 3.5

JAXB / JAXWS

Spring WS

XSLT Code Generation

EhCache 3

Springdoc OpenAPI 2.6

Micrometer Prometheus

OpenTelemetry

MapStruct 1.5

GitLab CI

Ansible

Docker / Jib

1 System Overview

A multi-module Maven project that exposes five independently deployable Spring Boot microservices (ports 9190–9194), each acting as a SOAP-to-REST proxy adapter. REST clients POST JSON payloads; each service converts them to SOAP XML via JAXB marshalling, calls the appropriate backend SOAP service using Apache CXF / Spring WS, and returns the response as JSON. REST controllers are entirely **auto-generated at build time** from WSDL files using an in-project XSLT code-generation engine — no hand-written proxy code required. A shared framework library provides the base adapter, authentication, caching, and observability concerns.

▲ REST Inbound Layer

- Auto-generated @RestController (per WSDL)
- @RequestMapping POST per SOAP operation
- @RequestHeader HTTP header extraction
- @RequestBody JSON → JAXB object (Jackson)
- @Timed Micrometer metric per endpoint
- Springdoc OpenAPI → Swagger UI

► Adapter / Translation Layer

- WebserviceProxyAdaptor (base class)
- JAXB Jaxb2Marshaller (JSON ↔ SOAP XML)
- HTTP header → SOAP MessageCallback
- CacheKeeper: SRD user SID lookup (EhCache)
- MDC logging (ContextId + User)
- SoapFaultClientException → JSON error

▼ SOAP Outbound Layer

- Apache CXF JAXWS client stubs
- Spring WS WebServiceTemplate
- LoggingClientInterceptor (SOAP envelope log)
- Logistics Query SOAP (port 9190)
- Logistics Update SOAP (port 9191)
- Search / Cost / Finance Settlement SOAPS

XSLT Code Generation Engine

GeneratorMain.java runs at generate-sources phase. Reads each .wsdl → extracts namespace, portType, operations → applies service-specific .xslt template → emits a fully compilable Java @RestController. No manual proxy code ever written.

Shared Framework Library

lib/architecture/framework (framework-titanwebuigateway) is a Maven dependency embedded in every service JAR. Provides WebserviceProxyAdaptor, CacheKeeper, LoggingClientInterceptor and Spring Boot auto-configuration for cache and metrics.

Header-Based Authentication

REST clients pass Username header. WebserviceProxyAdaptor calls CacheKeeper → Reference Data Service REST API to resolve Active Directory SID. SID is concatenated into the SOAP user header. EhCache caches SID per username for 1 hour.

Independent Service Topology

Five services each build as a separate Spring Boot fat JAR + Docker image via Jib. Each runs on its own port and is deployed, scaled, and versioned independently. No shared runtime state between services.

2 SOAP-to-REST Proxy Request Lifecycle



Key design note: The XSLT template hardcodes the Jaxb2Marshaller package path and WebServiceTemplate URL property key per service, so each generated controller is entirely self-contained — the code generator produces both the @RestController and the Spring WS wiring in a single pass.

3 XSLT Build-Time Code Generation

Phase 1 — generate-sources

exec-maven-plugin

GeneratorMain.java invoked. Reads packageMapping.dat to map each WSDL filename to a Java package and a Spring property key for the backend URL. Scans src/main/resources/wSDL/ for all .wsdl files.

Phase 2 — XSLT Transformation

JAXP / TransformerFactory

For each WSDL: parses XML DOM to extract targetNamespace, portType name, and all operation elements. Applies the matching .xslt template (one per service WSDL) via javax.xml.transform. Outputs Java source string.

Phase 3 — Compilation

fmt-maven-plugin + javac

Generated Java files written to target/generated-sources/titanweb/. google-java-format formats code. Maven compile phase picks up generated sources alongside hand-written Config.java. Final JAR contains all proxy classes.

4 Key Architecture Patterns

XSLT-Driven Code Generation (build-time)

WSDL files are the single source of truth. The GeneratorMain + XSLT templates produce @RestController Java classes during generate-sources. Adding a new SOAP operation requires only a WSDL change — no hand-written controller code, ever.

Adapter Pattern — WebserviceProxyAdaptor

All generated controllers extend a single base class that handles the entire cross-cutting concern stack: header extraction, SID resolution, JAXB marshalling, WebServiceTemplate call, error mapping, and MDC population. Each proxy method only defines its typed request/response.

Cache-Aside (EhCache 3) — User SID Lookup

CacheKeeper.getUserSidFromSRD() is annotated @Cacheable. EhCache stores username→SID mappings in-heap (10 entries) + off-heap (10 MB) with a 3600s TTL. Eliminates SRD round-trip on every request for active users.

Header Tunnelling — HTTP to SOAP

Authorization, ServiceCallContextId, ContextId, UserName, and DesktopVersion HTTP headers are forwarded into SOAP envelopes via WebServiceMessageCallback. Preserves end-to-end request context without modifying SOAP service contracts.

Protocol Error Normalisation

SoapFaultClientException caught in every generated proxy method and converted to a JSON ServiceResponsePayload with HTTP 400. REST consumers never see raw SOAP fault XML — they always receive structured JSON errors.

Shared Library via Maven Dependency

framework-titanwebuigateway is a Maven artifact published to the corporate registry. All five services depend on it identically. Framework upgrades (auth, caching, interceptors) propagate to all services with a single version bump.

@Timed per Operation (Micrometer)

Every generated proxy method is annotated @Timed(value="ServiceName_operationName"). TimedAspect bean (registered in framework) records latency histograms per SOAP operation. Metrics exposed at /actuator/prometheus.

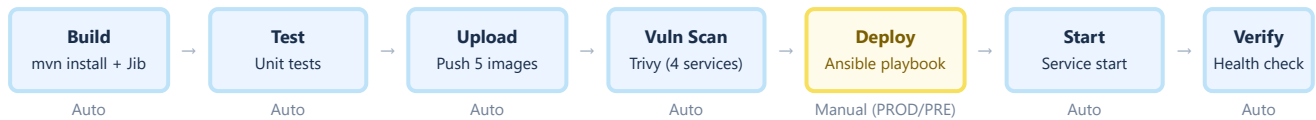
Static Username Fallback Mode

titan.static.username.enabled=true bypasses SRD lookup and uses a hardcoded username+SID. Used in environments where SRD is unavailable (e.g., DEV integration testing) without changing application logic.

5 Architecture Decisions

Decision	Choice	Rationale
Proxy controller generation	XSLT at build time (generate-sources)	WSDL is the authoritative contract. XSLT keeps 100% fidelity to WSDL operations — no drift between spec and implementation. Generated code is plain Java, fully debuggable.
SOAP client technology	Apache CXF 3.5 + Spring WS WebServiceTemplate	CXF provides mature WSDL→Java stub generation (cxf-codegen-plugin). Spring WS WebServiceTemplate adds interceptor support (logging) and Jaxb2Marshaller integration without boilerplate.
XML ↔ Object binding	JAXB (Jakarta XML Bind 4.0)	Required for Java 17 (javax.* removed). Jaxb2Marshaller configured per service with package path derived from packageMapping.dat — keeps marshaller scoped to each WSDL's generated stubs.
Date type handling	LocalDateAdapter + XMLGregorianCalendar adapters	SOAP services return XMLGregorianCalendar; REST clients expect ISO LocalDate/LocalDateTime. Per-service adapters convert at JAXB boundary — no date leakage into business logic.
Cache technology	EhCache 3 (Jakarta, JSR-107)	In-process, no infrastructure dependency. Heap + off-heap tiers for SID cache. Spring Cache abstraction means CacheKeeper logic is cache-implementation-agnostic.
Service topology	5 independent Spring Boot JARs	Allows individual service scaling, deployment, and versioning. Each SOAP domain (inventory, cost, index, cash) is isolated — a WSDL change in one service does not affect others.
Error contract	ServiceResponsePayload<T> wrapper	Uniform JSON envelope for both success and error responses. REST consumers parse a single contract regardless of SOAP fault type — decouples caller from SOAP protocol semantics.

6 CI/CD Pipeline



Each of the 5 proxy services is built as an independent Docker image and deployed separately. Trivy vulnerability scan runs on all active service images. PROD/PRE deployments require manual trigger. App version: 2.5.0.

7 Observability

Per-Operation Metrics (@Timed)

Every generated proxy method carries @Timed(value="ServiceName_operationName"). TimedAspect records call count, total time, and max latency histograms. Available at /actuator/prometheus per service.

Micrometer + Prometheus

Spring Boot Actuator exposes /actuator/prometheus, /actuator/health and /actuator/metrics endpoints. Micrometer 1.13.4 records JVM, HTTP, and SOAP proxy metrics. management.endpoints configured per service.

SOAP Envelope Logging

LoggingClientInterceptor is registered on every WebServiceTemplate. Logs full SOAP request and response envelopes at DEBUG level, including headers and faults. Enabled per environment via logback configuration.

MDC Correlation

WebserviceProxyAdaptor populates SLF4J MDC with CONTEXT_ID (ServiceCallContextId) and USER (UserName) on every request. Every log line carries these fields — correlates REST call with downstream SOAP call in Splunk.

Distributed Tracing

OpenTelemetry Java Agent 2.8.10 auto-instruments HTTP and Spring WS calls. Trace context propagated through the entire proxy chain: REST client → proxy service → SOAP backend. Exported to collector via OTLP.

Swagger / OpenAPI

Springdoc OpenAPI 2.6 auto-generates API documentation from each service's @RestController annotations. Swagger UI available at /swagger-ui.html per service port — driven entirely from generated controller code.

8 Technology Stack

